EDOS: Design and Implementation of a Browser-Based Python Learning Environment for Programming and Biological Data Science Education

Yash Munnalal Gupta

Department of Biology, Faculty of Science, Naresuan University, 99 Moo 9, Phitsanulok-Nakhonsawan Road, Phitsanulok 65000, Thailand

Corresponding author: yashmunnalalg@nu.ac.th

12 October 2026 | DOI: 10.5281/zenodo.23319351

Abstract

Practical Python instruction often requires learners and educators to configure development environments, manage dependencies, and distribute exercises before programming can begin. These requirements can complicate introductory teaching, especially in settings where installing local software is undesirable or restricted. This software design paper describes EDOS (Educational Desktop Operating Simulator), a browser-based educational desktop that brings several beginner-oriented programming activities into a single interface. Examination of the public repository identifies a Windows-inspired desktop, a Python editor and interactive terminal, Pyodide-based Python execution using WebAssembly, a Jupyter-style notebook interface, a browser-side package-management workflow, and a repository-backed learning library. The accompanying instructional materials include English and Thai teaching content, seven biology-oriented Jupyter notebooks, seven corresponding workshop PDF files, and three example CSV datasets. A biology data-analysis lesson illustrates sequence-length and GC-percentage calculations, species-level grouping, visualization, and simple treated-to-control gene-expression ratios using synthetic educational data. The available repository supports an architectural and content inventory, but does not establish educational effectiveness, cross-browser reliability,

performance, or learner outcomes. EDOS is therefore presented as an implemented educational software artifact and a foundation for future controlled usability and learning-outcome evaluation.

Keywords: Python education; browser-based programming; Pyodide; WebAssembly; notebook learning; biological data science

1. Introduction

Python is used across introductory programming, scientific computing, and biological data analysis. Scientific array programming has been described by Harris and colleagues in their publication on NumPy [4]. An instructional environment for new learners must support elementary syntax while allowing progression to tabular data processing and visualization. In conventional computer-laboratory workflows, software installation, interpreter configuration, and dependency management can introduce tasks unrelated to immediate learning objectives. Web-hosted programming environments represent one approach to simplifying access, although their practical capabilities depend on browser support, network conditions, and the compatibility of scientific libraries.

EDOS, short for Educational Desktop Operating Simulator, is a browser-based learning environment designed to resemble a familiar graphical desktop while exposing Python-focused educational applications [1]. The repository describes the intended users as students, teachers, coding clubs, workshop participants, and beginners in science and programming. Unlike a full operating-system emulator, the available code and documentation frame EDOS as a browser application with simulated desktop interactions and a client-side Python runtime.

This paper documents EDOS as a software-design artifact rather than an experimental intervention. Its objectives are to (i) describe the distribution and execution architecture, (ii) catalog the principal learning tools, (iii) characterize the included biology-focused instructional sequence, and (iv) identify evidence needed before claims can be made about educational outcomes or operational reliability.

2. Materials and Methods

2.1 Repository and review approach

The primary source was the public repository `yashmgupta/EDOS-Python-Learning-Desktop`, inspected on 12 October 2026 [1]. The default branch was `main`. Review materials included `README.md`, `ROADMAP.md`, `CHANGELOG.md`, `CITATION.cff`, the beginning of `index.html`, directory listings, teaching notebook sources, and tabular example data. The reviewed repository tree had Git tree identifier `c678725f0f3507bb916ffcd99e67d0c5c8e96bd8`. This identifier refers to a tree object, not necessarily the complete commit hash. This was a documentary and source inspection; no browser-based execution or participant study was performed for this manuscript.

2.2 Application architecture and delivery

EDOS is delivered as a browser application whose main entry point is `index.html`. The checked HTML establishes the application document, viewport settings, desktop-themed styling, and a configured external Pyodide script. The repository's README describes Python execution through Pyodide and WebAssembly, permitting supported Python code to execute inside the browser rather than requiring an application-specific remote Python execution service [1]. Pyodide is an established Python distribution for web browsers and WebAssembly environments [2]. The live deployment is documented at `https://yashmgupta.github.io/EDOS-Python-Learning-Desktop/`.

This client-side distinction does not imply that every activity works without connectivity. External JavaScript assets, first-time runtime acquisition, compatible Python packages, and live learning-library files may depend on network access. Neither offline performance nor third-party dependency availability was independently evaluated here.

2.3 Educational desktop and programming tools

The repository documentation identifies a Windows-inspired desktop interface, multiple application windows, a Python IDLE-style editing tool, and an interactive Python terminal. The documented notebook interface supports code and Markdown cells, per-cell or all-cell execution, a persistent session namespace, notebook saving, and inline plots when supported [1]. The implementation is Jupyter-style; it should not be confused with a claim that EDOS embeds the

standard Jupyter server or front end. The notebook paradigm is described by Kluyver and colleagues [3]. A package-management workflow is provided for Pyodide-compatible packages; packages with unsupported native operating-system dependencies cannot be assumed to function.

The stated teaching design combines guided lessons, beginner-friendly error explanations, English and Thai learning materials, PDF viewing, and a file-like workspace in the browser. These are reported as documented and/or source-associated features, not independently verified measures of accessibility or student usability.

2.4 Repository-backed learning library

EDOS organizes educational content under `book/`, with folders for Python materials, datasets, notebooks, and workshops. The README states that its Books/Live Library reads published GitHub-hosted resources and can expose updated materials without changing the main HTML page [1]. The folder organization separates reusable teaching documents from interactive Jupyter notebooks and example datasets. The documentation also restricts publication of private student data and unlicensed resources.

2.5 Biology-oriented notebook curriculum

The inspected repository tree includes seven numbered `.ipynb` notebooks and seven corresponding PDF workshop files. Their topics progress from Python and biology basics to DNA strings, collection types, sequence scanning, functions, classes, and CSV analysis. The first notebook uses small biology-context examples to introduce `print()`, variables, data types, and the `len()` function. The seventh notebook joins CSV loading, tabular inspection, reusable functions, filtering, grouping, plotting, and interpretation in a project-style exercise. Its code imports pandas for tabular operations [5] and Matplotlib for visualization [6]. NumPy is also introduced in repository documentation as a supported teaching-library example [4]; neither the availability nor exact version of these libraries was independently benchmarked in the EDOS runtime.

The latter notebook references `book/datasets/dna_samples.csv` and `book/datasets/gene_expression`
The DNA dataset contains six illustrative rows for *E. coli*, yeast, and *Arabidopsis* labels, while the gene-expression dataset contains six illustrative gene records. These are course examples, not evidence of an empirical biological experiment. The notebook explicitly describes its DNA

dataset as synthetic.

For an individual sequence, the notebook defines GC percentage as:

$$GC\,(\%) = \frac{N_G + N_C}{L} \times 100,$$

where N_G and N_C are counts of the respective bases and L is sequence length. The demonstrated workflow then filters records at a 50% threshold, computes mean GC percentages by species label, and constructs a bar chart. A separate exercise computes a simple ratio between `treated` and `control` columns:

$$\text{Fold ratio} = \frac{\text{treated}}{\text{control}}.$$

This teaching example is not a validated differential-expression pipeline. In particular, the notebook does not establish normalization, replication, uncertainty estimation, or inferential testing, and its basic GC function does not handle empty sequences.

2.6 Assessment scope

The present assessment is a structured feature and material inventory. No standardized usability instruments, pre/post knowledge tests, accessibility audits, security penetration tests, browser compatibility matrix, runtime benchmarks, or student-level outcome data were supplied or executed. Therefore, numerical claims about speed, learning gain, engagement, reliability, or comparative effectiveness are intentionally excluded.

3. Results

3.1 Identified software capabilities

The repository documentation and inspected application entry point establish EDOS as an integrated browser-based teaching artifact. Its described functional areas include (a) browser-side Python execution, (b) editing and terminal interactions, (c) notebook-style coding and written explanations, (d) compatible package loading, and (e) viewing and organizing educational materials. The `ROADMAP.md` file distinguishes these documented core features from incomplete items such as quizzes, a teacher mode, full accessibility work, and improved offline capabilities [1].

Table 1. Repository-based inventory of EDOS instructional artifacts.

Artifact	Inventory in reviewed tree	Educational role
Main web entry point	One `index.html`	Educational desktop application
Jupyter notebooks	Seven `.ipynb` files	Progression from basics to CSV biology analysis
Workshop handouts	Seven PDF files	Companion teaching material
CSV datasets	Three `.csv` files	Demonstration data for biology exercises
Main documentation	README, roadmap, changelog and policy files	Usage, development, and contribution guidance

3.2 Curriculum progression

The notebook file sequence supplies a clear structure for introductory instruction: basic syntax, DNA strings, lists and collections, conditions and loops, reusable functions, classes and objects, and finally CSV-based biology data analysis. The progression links general Python constructs to biological examples, thereby providing opportunities for domain contextualization. The existence of this sequence is verifiable; whether it improves completion rates or conceptual understanding remains unmeasured.

3.3 Data-analysis demonstration

The seventh notebook specifies loading a CSV into pandas, deriving sequence length and GC percentage, identifying a maximum-GC sample, filtering higher-GC samples, summarizing groups, and plotting GC values. It also demonstrates a simple gene-expression ratio calculation. The notebook file contains code and explanatory cells with empty recorded outputs; this review therefore reports the available analytical operations rather than presenting unexecuted calculations as experimental findings.

3.4 Limits of observed results

Inspection verified the presence and structure of the learning materials and the documented architecture. It did not verify actual rendering on all browsers, interoperability of every package,

the correctness of every user-interface interaction, or whether the system meets education or accessibility standards. Evidence for those outcomes requires execution and structured evaluation.

4. Discussion

The architecture of EDOS combines a familiar desktop metaphor with an in-browser Python runtime and a repository-managed curriculum. This may reduce some setup requirements for learners where a modern compatible browser is already available. However, an installation-free user interface should not be interpreted as network independence, unrestricted package compatibility, or complete parity with a local Python environment.

The project is particularly relevant as a demonstrator for introductory biological data analysis. Its sequence of notebooks bridges basic programming exercises and domain examples using DNA strings, GC percentage, CSV tables, and gene-expression ratios. Such examples may help instructors connect computational concepts to existing scientific curricula, but student comprehension, perceived usefulness, and transfer of skills must be assessed empirically.

Repository-managed instructional assets create a straightforward content publication workflow. However, this design also requires attention to content provenance, stable references, appropriate licensing, and changes to remote files over time. Recording a repository commit hash and versioned course materials would improve reproducibility across classroom sessions.

Several important limitations constrain the conclusions. First, the manuscript is based on source and documentation review rather than a controlled system test. Second, the teaching datasets are synthetic examples rather than independently collected scientific observations. Third, teaching content does not by itself demonstrate learning outcomes. Fourth, planned features in the roadmap are not current implemented capabilities. Future work should include browser-compatibility testing, deterministic notebook execution checks, accessibility audits, package-loading reliability tests, and student-centered evaluations using predefined learning objectives and appropriate ethics safeguards.

5. Conclusion

EDOS is a browser-based educational desktop project integrating Python execution, editing, notebook workflows, and learning resources into a single programming environment. The re-

viewed repository includes a coherent set of introductory biology-oriented notebooks, accompanying PDF handouts, and example datasets. These materials demonstrate a concrete software artifact that could support domain-contextualized Python teaching. At present, the evidence supports descriptive claims about the repository's implementation and educational content, but not claims that EDOS improves achievement, usability, or access compared with alternative learning environments. Prospective classroom testing and independently reproducible technical evaluations are the next steps toward a stronger educational research assessment.

Conflicts of Interest

The author developed EDOS and has a direct interest in its dissemination.

Data and Code Availability

Source code, educational notebooks, workshop documents, and sample datasets are available in the public project repository: https://github.com/yashmgupta/EDOS-Python-Learning-Desktop The reviewed tree identifier was `c678725f0f3507bb916ffcd99e67d0c5c8e96bd8`; a full commit SHA should be recorded for a final reproducibility statement. The main live page is https://yashmgupta.github.io/EDOS-Python-Learning-Desktop/ . The repository uses a custom source-visible license; public accessibility should not be assumed to imply an unrestricted open-source license.

References

[1] Y. M. Gupta, *EDOS — Educational Desktop Operating Simulator*, software repository, GitHub, 2026. Available: https://github.com/yashmgupta/EDOS-Python-Learning-Desktop (accessed 12 October 2026).

[2] The Pyodide Development Team, *Pyodide: Python with the scientific stack, compiled to WebAssembly*, project documentation. Available: https://pyodide.org/en/latest/project/about.html (accessed 12 October 2026). A version-specific archival software citation should be added if the production runtime version is confirmed.

[3] T. Kluyver *et al.*, "Jupyter Notebooks—a publishing format for reproducible computational workflows," in *Positioning and Power in Academic Publishing: Players, Agents and Agendas*, 2016, pp. 87–90. doi: 10.3233/978-1-61499-649-1-87.

[4] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020. doi: 10.1038/s41586-020-2649-2.

[5] W. McKinney, "Data structures for statistical computing in Python," in *Proceedings of the 9th Python in Science Conference*, 2010, pp. 56–61. doi: 10.25080/Majora-92bf1922-00a.

[6] J. D. Hunter, "Matplotlib: A 2D graphics environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007. doi: 10.1109/MCSE.2007.55.